Cheat sheet

Structure du programme

```
#include <stdio.h>
int main() {
    //Ecrire le code ici
}
```

Variables : définitions

Un nom, l'adresse de la première case de l'espace mémoire réservé, un type.

Variables : déclarations

```
type nomVariable = valeur;
```

```
double temperature ;
int age = 20;  // avec initialisation
```

Variables : types

- **int** : entier
- **double** : décimal
- **bool** : vrai/faux

```
int age = 25;
double prix = 19.99;
bool estVrai = true;
```

Entrées / sorties

Entrée :

```
scanf("%typevar", &nomvar);
//typevar : %d = int, bool | %lf =
double
```

Sortie :

```
printf("Texte avant %typevar texte
après.\n");
// %typevar :  %d = int, bool | %f =
double
```

`\n` passe à la ligne

Conditions : opérateurs comparatifs

`a == b`	a **égal** b
`a != b`	a **différent** de b
`a >b`	a **strictement supérieur** à b
`a >= b`	a **supérieur ou égal** à b

Conditions : structure

```
if (condition) { // si
    // exécuté si condition est vrai
}
else if (autreCondition)  //sinon si
{
    // exécuté si condition est faux et
autreCondition est vraie
}
else //sinon
{
    // exécuté si aucune condition est
vraie
}
```

Conditions : opérateurs logiques

`&&` ET logique

```
if ( a > 5 && b < 10)
```

`||` Ou logique

```
if ( a == 3 || b == 7)
```

`!` NON logique

```
if ( ! estVrai )
```

Fonction : définition

```
type_retour nom_fonction(type param1,
type param2, ...)
{
    // instructions
    return valeur;
}
```

De zéro à autant de paramètres que souhaité.
Si le type de retour est `void` : la fonction ne retourne rien.
`return` déclenche le retour d'une valeur et quitte la fonction immédiatement.

Fonction : appel

```
int resultat = addition(5, 3);
```

`addition` prend `5` et `3` comme paramètre, va retourner `8` qui va être stocké dans `resultat`.

Fonction : passage par valeur / copie

Par défaut, les arguments sont passés par valeur (une copie est faite). Modifier une variable dans la fonction n'affecte pas la variable d'origine.

Boucle : for

Boucle définie, quand nous savons au moment de commencer la boucle combien de tours on va faire.

```
for (init; condition;
incrémentation)
{
    // Code à exécuter
}

Exemple :
for (int i = 0; i < 10; i=i+1)
{
    printf("%d\n", i);
}
// se lit : pour i de 0 à 9
```

Boucle : while

Boucle indéfinie, quand nous ne savons pas combien de tours nous allons faire au moment de commencer la boucle.

```
while (condition) // tant que
{
    // Code à exécuter
    // tant que la condition est
vraie
}

Exemple :
int i = 0;
while (i < 10)
{
    printf("%d\n", i);
    i++;
}
```

Fonction : surcharge

Deux fonctions peuvent s'appeler pareil si les arguments sont différents.

Fonction : où les écrire ?

- Prototypes : `.hpp`
- Corps : `.cpp`

Penser à `#include "fichier.hpp";` dans le `.cpp` et dans chaque fichier où l'on souhaite utiliser la fonction.

?

Pointeurs : déclaration et initialisation

Un pointeur est une variable qui contient l'adresse d'un autre espace mémoire et permet d'y accéder.

```
type * pNom ;
pVar = & var; //met l'adresse de var
dans le pointeur
```

Pointeurs : utilisation

```
printf("%p\n", pVar); //adresse pointée
printf("%d\n", *pVar); // valeur
stockée à l'adresse pointée
```

Ne pas confondre * à la déclaration d'un pointeur et à son utilisation !

Pointeurs : fonctions

Permet de dépasser la portée des variables et de retourner plus d'un élément.

```
type_retour maFonction ( type *
pPointeur )
```

pPointeur pointe vers un espace mémoire donné.
Que ça soit avant la fonction ou dans la fonction, c'est le même.
Donc si la valeur pointée est modifiée elle l'est aussi à l'extérieur !

Allocation dynamique

- `sizeof` : donne la taille nécessaire en mémoire pour un type de variable
- `malloc` : alloue un espace mémoire.
- `free` : désalloue (libère) l'espace.

```
type * pPointeur = malloc( sizeof(type)
);
type * tab =  malloc( sizeof(type) *
size);
NomStructure instance = malloc
(sizeof(NomStructure ));
free var;
```

Tableau: définition

Une série d'espaces mémoires contiguës (côte à côte) qui contient chacun une variable.
Un tableau contient l'adresse du premier espace et à besoin de connaître le type.
Attention : il ne sauvegarde pas sa taille ! Il faut garder l'information stockée ailleurs.

Tableau : déclaration

```
type nomTableau [ 300 ] ;
```

La taille n'est pas une variable mais une valeur directement écrite dans le code (ici pour l'exemple 300).

Tableau : initialisation à la déclaration

Complète (toutes les valeurs sont données entre {}) :

```
int tab [ 3] = {3,4,5};
```

Partielle (seules les premières valeurs sont données entre {}) :

```
int tab [ 5 ] = {8, 1} ;
```

Le reste est rempli automatiquement par des 0.

Tableau : utilisation

La numérotation d'un tableau commence à 0.

```
int tab[15];
tab ; // adresse de la première case
tab[0] ; //première case
tab[5]; // sixième case
tab[14] ;  // 15ième et dernière case
tab[15]; // on a dépasser la taille...
erreur !
```

Attention à ne pas dépasser la taille du tableau !

Tableau et fonctions

Le tableau ne contient pas sa taille, il faut donc passer la taille en plus dans une fonction !

```
type_retour maFonction ( type tab[],
int taille)
```

Retourner un tableau

```
typeTableau * maFonction ( )
```

Tableau : 2D (Matrice)

```
type nomTab [ 8 ][ 5 ] ;

int tab [ 8 ][ 3 ] = { { 3 , 2, 1 } , {
8 , 4 , 3 } , ...};

printf("%d\n", tab [ 1 ] [ 0 ]); //
affiche 8

type_retour maFonction ( type tab[],
int tailleX, int tailleY)
```

Classe : définition

Regroupe des attributs (données) et des méthodes (fonctions) en une seule entité.

Classe : privé / public

Qui peut y accèder (en lecture et écriture) ?

- `private` : les fonctions de la classe.
- `public` : tout.

Structure : définir

Dans un fichier NomStructure.hpp vous allez déclarer le squelette de la structure

```
typedef struct
{
    type member1; // donnée
membre
    type member2;
}  NomStructure ; // Ne pas
oublier le ; .

//fonctions manipulant la
structure
type maFonction ( mesparametres)
;
type maFonction2 (
mesparamètres) ;
```

Structure : implémentation des fonctions liées

Dans un fichier Structure.cpp

```
#include "NomClasse.hpp"
type  maFonction (
mesparametres)
{
    // code de la fonction
}

type  maFonction2 (
mesparametres)
{
    // code de la fonction
}
```

Structure : instance

Une instance est une variable dont le type est la structure choisie. (Pour les fonctions)

```
NomStructure nomInstance;
```

Structure : initialisation

```
NomStructure nomInstance = {120;
120}; //sequentielle
NomStructure nomInstance =
{.member2 = 120}; //sélective
```

Si sélective, les données non initialisées sont mises à `0` par défaut. Et les pointeurs à `NULL`.

↳ structure.variable
= recup variable

(différent de :
structure -> variable
(*structure).variable
+ &variable -> adresse variable structure
?

BASH SCRIPTING CHEATSHEET

#General

Hello world

```
#!/usr/bin/env bash

NAME="world"
echo "Hello ! $NAME"
```

Variables

```
NAME="John"

echo $NAME
echo "$NAME"
echo "${NAME}"
```

Conditional execution

```
git commit && git push
git commit || echo "Commit failed"
```

String quotes

```
NAME="John"

echo "Hi $NAME" #=> Hi John
echo 'Hi $NAME' #=> Hi $NAME
```

Shell execution

```
echo "I'm in $(pwd)"
# Or
echo "I'm in `pwd`"
```

Conditionals

```
if [[ -z "$string" ]]
then
    echo "String is empty"
elif [[ -n "$string" ]]
then
    echo "String is not empty"
fi
```

#Loops

Basic for loop

```
for i in `ls /etc/rc.*`
do
    echo $i;
done
```

C-Like for loop

```
for ((i = 0 ; i < 100 ; i++))
do
    echo $i;
done
```

Ranges

```
for i in `seq 1 1 5`
do
    echo "$i";
done
```

#Functions

Defining functions

```
function myfunc() {
    echo "hello $1 »
}

myfunc "John"
```

Returning values

```
function myfunc() {
  local myresult="some value"
  echo $myresult
}

result="$(myfunc)"
```

Arguments

$#	Number of arguments
$*	All positional arguments (as a single word)
$@	All positional arguments (as separate strings)
$1	First argument
$_	Last argument of the previous command

#Array

Defining arrays

```
Fruits=("Apple" "Banana" "Orange")
# Or
Fruits[0]="Apple"
Fruits[1]="Banana"
Fruits[2]="Orange"
```

Iteration

```
for i in "${arrayName[@]}"
do
    echo $i
done
```

Working with arrays

```
echo ${Fruits[0]}      # Element #0
echo ${Fruits[-1]}     # Last element
echo ${Fruits[@]}      # All elements, space-separated
echo ${#Fruits[@]}     # Number of elements
echo ${#Fruits}        # String length of the 1st element
echo ${#Fruits[3]}     # String length of the Nth element
echo ${Fruits[@]:3:2}  # Range (from position 3, length 2)
echo ${!Fruits[@]}     # Keys of all elements, space-separated
```

LINUX COMMAND CHEATSHEET

Directory operations

pwd	Show current directory
mkdir *dir*	Make directory *dir*
cd *dir*	Change directory to *dir*
ls	List file

IO Redirection

cmd < file	Input of *cmd* from *file*
cmd > file	Standard ouput (stdout) of *cmd* to *file*
cmd >> file	Append stdout to *file*
cmd* 2> *file	Error output (stderr) of *cmd* to *file*
cmd 1>&2 file	Stdout to same place as stderr
cmd 2>&1 file	Stderr to same place as stdout

Command lists

cmd1 ; cmd2	Run cmd1 then cmd2
cmd1 && cmd2	Run cmd2 if cmd1 is successful
cmd1 || cmd2	Run cmd2 if cmd1 is not successful
cmd &	Run cmd in a subshell

File permission numbers

First digit is **owner** permission, second is **group** and third is **everyone**.

Calculate permission digits by adding numbers below.	
4	read (r)
2	write (w)
1	executre (x)

Exemples :
chmod 777 - read, write, execute for all
chmod 755 - rwx for owner, rx for group and world

Process management

ps	Show snapshot of processes
top	Show real time processes
kill *pid*	Kill process with id process
pkill *name*	Kill process with name *name*
killall *name*	Kill all process with names beginning *name*

Bash variable

env	Show environment variables
echo $NAME	Output value of $NAME
export NAME=value	Set $NAME to value
$PATH	Executable search path
$HOME	Home directory

File operations

touch file	Create *file*
cat file	Display *file*
cp file1 file2	Copy *file1* to *file2*
mv *file1 file2*	Move *file1* to *file2*
rm *file1*	Delete *file1*
head *file1*	Show first 10 lines of *file1*
tail *file1*	Show last 10 lines of *file1*

Text processing

cat	Concatenate files and print stdout
sort	Sort lines of text files
uniq	Report or omit repeated lines
cut	Remove section from each line of files
paste	Merge lines of files
join	Join lines of two files on a command field
tr	Translate
sed	Stream editor
wc	Count number of line, word, and bytes

Searching

grep pattern files	Search for pattern in files
grep –r pattern dir	Search recursively for pattern in dir
command | grep pattern	Search for pattern in the ouput of command